Frontend Developer — Step 6: Learn Tools

Expanded notes: Git, GitHub, branches, npm, build tools, DevTools, environment variables, CORS and security

1. Git Fundamentals

Definition

Git is a distributed version-control system used to track source-code history and collaborate safely.

Core terms

Repository = project history. Commit = recorded snapshot. Branch = independent development line. Merge = combine histories. Remote = hosted repository reference.

```
git init
git status
git add .
git commit -m "Initial commit"
git log
```

2. Branches & Collaboration

Commands

```
git switch -c feature-careers
git switch main
git merge feature-careers
```

Why branches?

Develop features or fixes without directly changing stable main code.

Pull request flow

Branch → implement → commit → push → pull request → review → merge → update local main.

3. GitHub

Definition

GitHub hosts Git repositories and provides collaboration features such as pull requests, issues and code review.

```
git remote add origin <repository-url>
git push -u origin main
git pull origin main
```

Security

Never commit passwords, private API keys, database credentials or secret .env files.

4. Package Managers & npm

Definition

A package manager installs, updates and tracks dependencies and can run project scripts.

```
npm init -y
npm install package-name
npm install -D package-name
npm run dev
npm run build
```

package.json

Contains metadata, dependencies and scripts. package-lock.json records resolved dependency versions/tree for reproducible installs.

5. Build Tools

Definition

Build tools transform, bundle and optimize source code for development or production.

Pipeline

Source → dependency resolution → transformation → bundling → optimization → production assets.

Development vs production

Development focuses on fast feedback; production builds focus on optimized deployable assets.

6. Browser Developer Tools

Console

Inspect JavaScript errors, warnings and logs.

Elements

Inspect HTML/CSS, computed styles, box model and responsive layout.

Network

Inspect request URL, method, status, headers, payload, response and timing.

Application

Inspect localStorage, sessionStorage, cookies and browser storage.

```
Network debugging:
1. Open DevTools → Network
2. Perform the action
3. Select request
4. Check status
5. Inspect request/response
6. Fix using evidence
```

7. Environment Variables, CORS & Security

Environment variable definition

Environment variables store configuration separately from source code and allow different values per environment.

```
DATABASE_URL=...
SECRET_KEY=...
API_KEY=...
```

Critical security rule

Anything shipped to frontend JavaScript can be inspected by users. Therefore frontend-exposed values are not secrets.

CORS definition

Cross-Origin Resource Sharing controls which browser origins can access a server's resources. Configure allowed origins deliberately in production.

8. Formatting, Linting & Debugging

Formatter

Keeps code formatting consistent.

Linter

Detects potential errors and problematic patterns.

Debugger

Supports breakpoints and step-by-step execution.

Workflow

Read error → identify file/line → reproduce → inspect → smallest fix → retest.

9. Professional Workflow & Checklist

```
1. Pull latest code
2. Create feature branch
3. Implement small change
4. Test/build
5. Check Console + Network
6. Commit clearly
7. Push
8. Review/merge
9. Pull main
```

Checklist

Git ✓ GitHub ✓ Branches ✓ Commits ✓ PRs ✓ npm ✓ package.json ✓ Build ✓ DevTools ✓ Network ✓ Env vars ✓ Security ✓ Debugging.